Master Team Project Summer 2024
ClassCompanion
Milestone 1

Team 1

Yagubkhan Yagubkhanov, Team Lead / Github Master
yagubkhan.yagubkhanov@informatik.hs-fulda.de

Samman Adhikari, Frontend Lead

Shantanu Patil, Backend Lead

Aralikere Yogananda Gunarachana

Mariha Asif

Amber Asad

Date: 17.05.2024
Date Submitted: 13.05.2024
Date Revised: 17.05.2024

Revision Summary:

Date	Revision	Summary
13.05.2024	1.0	Initial draft created
17.05.2024	1.1	Revised use cases and competitive analysis, added unique features

1.Executive Summary:

The motivation behind the development of our application is found in the crucial need for accessible and effective academic support within the Fulda University of Applied Sciences community. With a diverse range of users, including students, tutors, parents, administrators, and moderators, a seamless and secure environment is aimed at connecting students with tutors, facilitating learning, and fostering collaboration. By offering tailored services, ensuring user privacy, and adhering to established security protocols, the academic experience is aimed at being enhanced, and academic success for

students is aimed at being promoted while also streamlining administrative tasks for educators and administrators.
A comprehensive suite of functions and services tailored to the needs of our diverse user base will be provided by our application. Students will be able to create profiles, search for tutors based on various criteria, engage in live chat sessions, and provide feedback through ratings and reviews. Tutors, on the other hand, will have the ability to create detailed profiles showcasing their expertise, offer tutoring services, and communicate with students. Tools for managing user accounts, content moderation, and ensuring platform integrity will be available for administrators. By prioritizing ease of use, security, and scalability, a vibrant learning community is aimed at being fostered that promotes knowledge sharing and academic growth while providing essential support for all stakeholders within the university ecosystem.
Our student startup team comprises six members, each bringing a unique blend of experience and fresh perspectives to the table. While most of us have significant experience in the industry, a few are new to it, creating a balanced mix of seasoned professionals and fresh talent. This diversity allows us to approach challenges from different angles, drawing on both the knowledge gained from years of industry experience and the innovative ideas brought by newcomers. Together, we form a dynamic team capable of tackling complex problems and driving our startup forward with creativity and expertise.

2. **Personae and main Use Cases**:

Personas

Persona 1 (Student)
John Smith is a student at Hoschule Fulda and he is pursuing Bachelors of Science in Applied Computer Science. Besides his regular lecture and self study, he often finds himself struggling with concepts and needing someone else's help. So, to make sure he no longer wastes his time, he has decided to seek some professional help. He is now looking for some tutor online who can provide him direct one to one guidance.
Goals:

- Discover Computer Science Tutors: John wants to find some tutors online who can help him with his computer science studies.
- Convenient Communication: Johns was a very convenient platform to connect to those tutors and share resources and ideas.

Pain Points:

- Low Budget: Being a student, John is tight on budget.
- Concerns About Finding A Suitable Tutor: Since every tutor has his/her own style, finding a tutor who can match John’s style of learning can be difficult.

Persona 2 (Tutor)

Sara Brown is a professional Software Engineer, with around 2 years of experience. Besides building software, Sara also loves to teach. So, she has decided to become an online tutor for computer science students, where she will be able to earn some extra cash doing what she loves.

Goals:

- Market Herself As A Tutor: Sara wants to showcase her skills and market herself, so that she can reach students who she will be able to help.
- Convenient Communication: Sara also wants a convenient platform to connect to her students.

Pain Points:

- Limited Time: Since Sara has a full time job, she has very limited time.

Use Cases

1. **Discover suitable tutor - for John Smith (student)**
   John will login to the application and directly type in the course name that he wants help with. He will also apply filters, such as years of experience, languages etc. Then, he will see and evaluate the listed tutors based on quick information like the rating, price (hourly/per question) etc. If John sees any tutor that piques his interest, he will click on that tutor's link to open the details page. There he will see details such as the tutor's bio, reviews etc. Then John will be able to send a request to connect, including an initial message to the tutor.

2. **Create your profile - for Sara Brown (tutor)**
   Sara will come to our application to register as a tutor. She will first enter basic information like their name, email address etc. Then she will enter some more information like bio, their field of expertise, years of experience etc. This additional information will be crucial to market Sara to suitable students. Then finally Sara will be able to enter her desired hourly class rate and maybe even query rates (for example 10 query answers through text chat for x euros). Now, she will just wait for students to send her connection requests.

3. **Ask tutor's help - for John Smith (student)**
   John has a problem that he needs help with. He will go to his chat page. There John will message the tutor directly asking help for the question he has. John will then wait for the tutor to answer his question.

4. **Guest access - for Martin Wilson (guest)**
   Martin, a new interested student, first opens our application. He will have the option to wander in a few pages of the app without registering. Martin will first be greeted with some of the highest rated tutors of different fields. He will also be able to search for a tutor of any specific area (e.g. Maths). Martin will then see a

list of highest rated math tutors. In the same list he will also see important information like their rating, quick bio etc. Then when Martin finds a suitable tutor and clicks on it, he will be redirected to the registration page.

3.**List of main data items and entities**:

1. Type of Users:

Student: A registered user looking for tutoring assistance.
Tutor: People who provide tutoring services.
Administrator: In charge of overseeing and approving the profiles of tutors and keeping the website up to date.
Guest: Non-registered visitors can view tutor profiles and obtain basic service information.
Parent/Guardian: Individuals who oversee student accounts; particularly pertinent for students who are not yet of legal age.
Moderators: Individuals in charge of keeping an eye on and overseeing user-generated material, including comments and reviews.
Fulda Representative: A person from Fulda University of Applied Sciences who has been granted unique access or privileges on the site, allowing them to post tutor job openings or register many students at once.

2. Student/Tutor Profile:

Comprehensive information about all the registered students and tutors using the website including personal information, contact details, qualifications, experiences, academic details, and preferences.

3. Messages:

Students will be able to communicate with tutors and each other.

4. Available Services:

Complete information about all the courses offered, and their fees, duration, availability of tutors etc.

5. Feedback:

Student ratings of tutors and reviews about their tutoring experiences.

6. Administration:

Features that are provided for website administrator supervise tutor profiles, reviews, user accounts, and platform content quality.

7. Search Bar:

By simply typing keywords, students will be able to look for and select teachers by subject, availability, rating, etc.

8. Multimedia:
Tutors can upload their CV, files, lectures after administrator approval.

4.**Initial list of functional requirements**:

1. **User Registration and Login:**
    - Students and tutors must register using their university email address.
    - Check box to register as student or tutor
    - Registration process should include validation of email addresses to ensure eligibility.
    - Registered users (students and tutors) should be able to log in and log out securely.
2. **Profile Management:**
    - Users (students and tutors) should be able to manage their profiles, including updating personal information, adding skills, and uploading profile pictures.
    - Students should not be able to search for other students based on skills or subjects in tutor search.
2. **Tutor Search Functionality:**
    - Students should be able to search for tutors based on skills, subjects, or keywords.
    - Search results should display a list of relevant tutors based on the search criteria.
2. **Live Chat Feature:**
    - Students should be able to initiate live chat sessions with tutors they want to contact.
    - Chat functionality should support text messages, file uploads (such as documents or images), and other multimedia content.
    - If a tutor is offline, messages should be delivered to their inbox for later viewing.
    - Group chat so a teacher can connect with multiple students interested in same topic simultaneously
2. **Inbox Management:**
    - Both tutors and students should have access to their respective inboxes.
    - Inboxes should display messages received from other users, including chat messages and notifications.
    - Users should be able to mark messages as read, reply to messages, and delete messages as needed.
2. **Guest Access:**

- Guest students and teachers should be allowed to access the website, but they cannot utilize chat functionality until they register with their university email address.

2. **Admin Panel:**
   - An admin panel should be available for administrators to manage user accounts, monitor chat activity, and handle any reported issues.
   - Admins should have the ability to view user profiles, moderate content, and enforce site policies.

5.List of non-functional requirements:

Mandatory Requirements:

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server.
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers.
3. All or selected application functions must render well on mobile devices.
4. Data shall be stored in the database on the team's deployment cloud server.
5. No more than 50 concurrent users shall be accessing the application at any time.
6. Privacy of users shall be protected and all privacy policies will be appropriately communicated to the users.
7. The language used shall be English (no localization needed).
8. Application shall be very easy to use and intuitive.
9. Application should follow established architecture patterns.
10. Application code and its repository shall be easy to inspect and maintain.
11. Google analytics shall be used (optional for Fulda teams).
12. No e-mail clients shall be allowed.
13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
14. Site security: basic best practices shall be applied (as covered in the class) for main data items.
15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today.
16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development.
17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0.
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering

Project, Fall 2021 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application).

1. Performance:

Response Time: To provide a seamless user experience, all platform activities, including finding tutors, loading tutor profiles, and sending questions, should take less than two seconds to complete.
Scalability: To handle growing user and data volumes without noticeably degrading performance, the system should be built to scale horizontally. This includes the capacity to increase the number of server instances by using cloud-based elastic scaling options.

Resource Utilization: To ensure smooth operation even under heavy loads, the application should be optimized to make the best use of the system's resources. This includes minimizing CPU and memory utilization.

2. Expected Load:

Peak Load Handling: To avoid downtime or performance difficulties, the platform must be able to handle peak loads during exam seasons, registration periods, and other periods of high demand.
Load Testing: To find bottlenecks and adjust system performance appropriately, load testing should be done regularly.

3. Security Requirements:

Data Encryption: To avoid unwanted access, all sensitive user data, such as payment card numbers and personal information, should be encrypted both in transit and at rest.
Authentication procedures: To guarantee that only authorized users may access the platform, strong authentication procedures like multi-factor authentication should be put in place.
Authorization: To limit access to sensitive features or data based on user roles, role-based access control should be implemented.
Data privacy: It is important to guarantee adherence to data protection laws and to establish explicit guidelines for the handling, storing, and sharing of data.

4. Storage:

Data Retention: To guarantee legal compliance and maximize storage utilization, establish policies for data retention and archiving.
Backup and recovery: To guard against losing important data in the event of a cyberattack, hardware malfunction, or unintentional deletion, regular backups of that data should be made.
Scalable Storage: Without compromising performance or dependability, the storage system should be scalable to handle increasing data volumes over time.

5. Availability:

High Availability: To reduce downtime and guarantee availability of at least 99.9%, the platform should be built with redundancy and failover techniques.

Monitoring and Alerting: It is recommended to establish continuous monitoring of the system's uptime, performance metrics, and health. If any anomalies or errors are detected, an automatic alarm system should be set up.

6. Fault Tolerance:

Fault Detection: Implement mechanisms for detecting faults and errors in real-time, such as automated error logging and monitoring.
Fault Isolation: Isolate faults to prevent cascading failures that could affect other parts of the system, ensuring that issues are contained and localized.
Graceful Degradation: Design the system to gracefully degrade functionality in the event of failures, prioritizing core features and ensuring that users can still access essential services even during outages.

6.**Competitive analysis**:

Feature	**Competitor 1 (Tutor.com)**	**Competitor 2 (Chegg Tutors)**	**Competitor 3 (Wyzant)**	**Our Planned Product**
Registration Process	Email validation required	Email validation required	Email validation required	Email validation required
Live Chat	YES	YES	YES	YES
File Sharing	YES	YES	YES	YES
Profile Management	Limited	Limited	Limited	YES
Admin Panel	YES	YES	YES	YES
Group Chats	NO	NO	NO	YES
Calendar Integration	Limited	Limited	Limited	YES

Our planned product aims to offer a comprehensive online tutoring platform with features similar to those of competitors such as Tutor.com, Chegg Tutors, and Wyzant. However, our platform distinguishes itself by requiring users to register using their university email address from HS Fulda, ensuring a more secure and tailored experience within the university community. Additionally, our platform prioritizes user-friendly profile

management, allowing users to update personal information, add skills, and upload profile pictures easily.

We further enhance our platform with unique features such as Group Chat, enabling tutors and multiple students to discuss complex ideas and topics collaboratively. Our Calendar Integration feature allows tutors to show their availability, making it easy for students to schedule sessions. This includes creating meeting links and inviting all participants once a meeting is scheduled, ensuring everyone stays organized and connected.

The inclusion of a robust admin panel also sets our product apart, providing administrators with tools to manage user accounts, monitor chat activity, and enforce site policies effectively. Overall, our planned product aims to offer a seamless and efficient online tutoring experience, tailored specifically to the needs of the HS Fulda community.

7.High-level system architecture and technologies used:

Introduction

This section outlines the high-level system architecture and the technologies chosen for the development of our tutoring service website. The selection of these technologies is aimed at ensuring scalability, security, and performance while providing a seamless user experience.

Technologies Used

Backend Framework: Django
Description: Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the Model-View-Controller (MVC) architectural pattern, providing robust tools for building web applications.
Justification: Django's built-in features such as authentication, ORM (Object-Relational Mapping), and admin panel make it well-suited to our project requirements.

Frontend Framework: React
Description: React is a JavaScript library for building user interfaces, developed by Facebook. It allows developers to create reusable UI components and efficiently manage the state of the application.
Justification: React's component-based architecture facilitates the creation of dynamic and interactive user interfaces, enhancing the overall user experience. Its virtual DOM (Document Object Model) enables efficient rendering, resulting in faster performance.

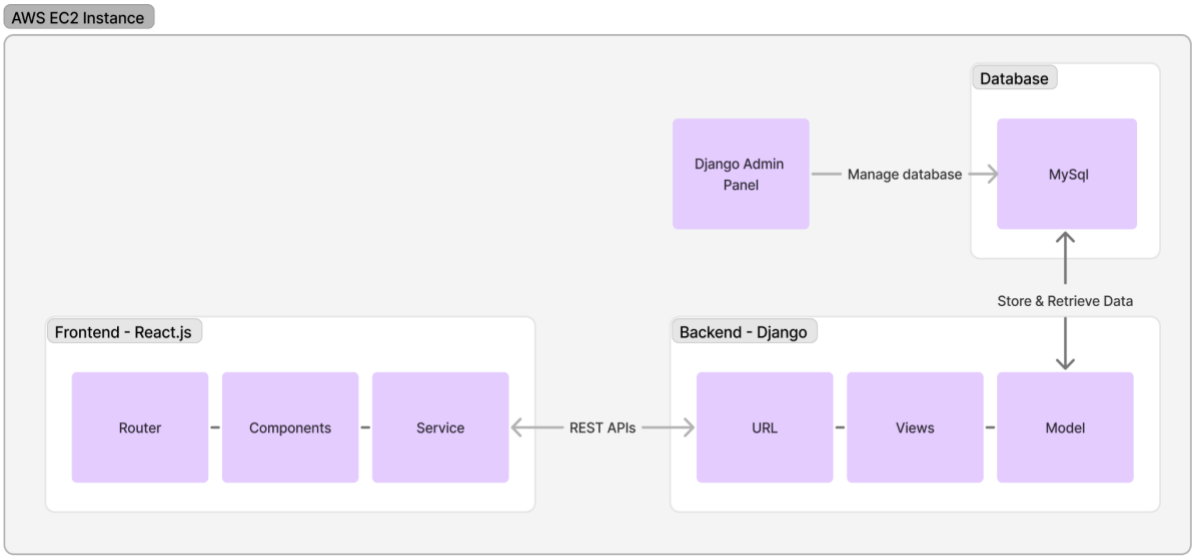
Database Management System: MySQL
Description: MySQL is an open-source relational database management system (RDBMS) known for its reliability, scalability, and performance. It provides robust features for managing structured data.
Justification: MySQL's widespread adoption, strong community support, and compatibility with Django make it an ideal choice for storing and managing various data entities in our application, including user profiles, messages, and feedback.

Cloud Platform: AWS (Amazon Web Services)
Description: AWS is a cloud computing platform offering a wide range of services, including computing power, storage, and databases, delivered as on-demand services over the internet.
Justification: AWS provides scalable and reliable infrastructure services, allowing us to deploy and manage our application with ease. Its comprehensive set of cloud services, including Amazon EC2 for hosting and Amazon RDS for MySQL database hosting, meets our project's requirements for scalability and reliability without any charges.

Additional Tools and Services

GitHub
Description: GitHub is a web-based platform for version control and collaboration, widely used by developers to host and review code, manage projects, and build software.
Justification: GitHub serves as our central repository for storing project code, enabling collaborative development among team members. Its version control features facilitate code management, branching, and merging, ensuring code quality and integrity.

Architecture Diagram



8.**Team and roles**:

Yagubkhan Yagubkhanov: Team Lead / Github Master / Document Master
Samman Adhikari: Frontend Lead
Shantanu Patil: Backend Lead
Aralikere Yogananda Gunarachana: Frontend
Mariha Asif: Frontend
Amber Asad: Backend

9. **Checklist**:

- Team found a time slot to meet (online) outside of the class: **DONE**
- GitHub master chosen: **DONE**
- Team decided and agreed together on using the listed SW tools and deployment server: **DONE**
- Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing: **DONE**
- Team lead ensured that all team members read the final M1 and agree/understand it before submission: **DONE**
- GitHub organized as discussed in class (e.g. master branch, development branch, folder for milestone documents etc.): **DONE**